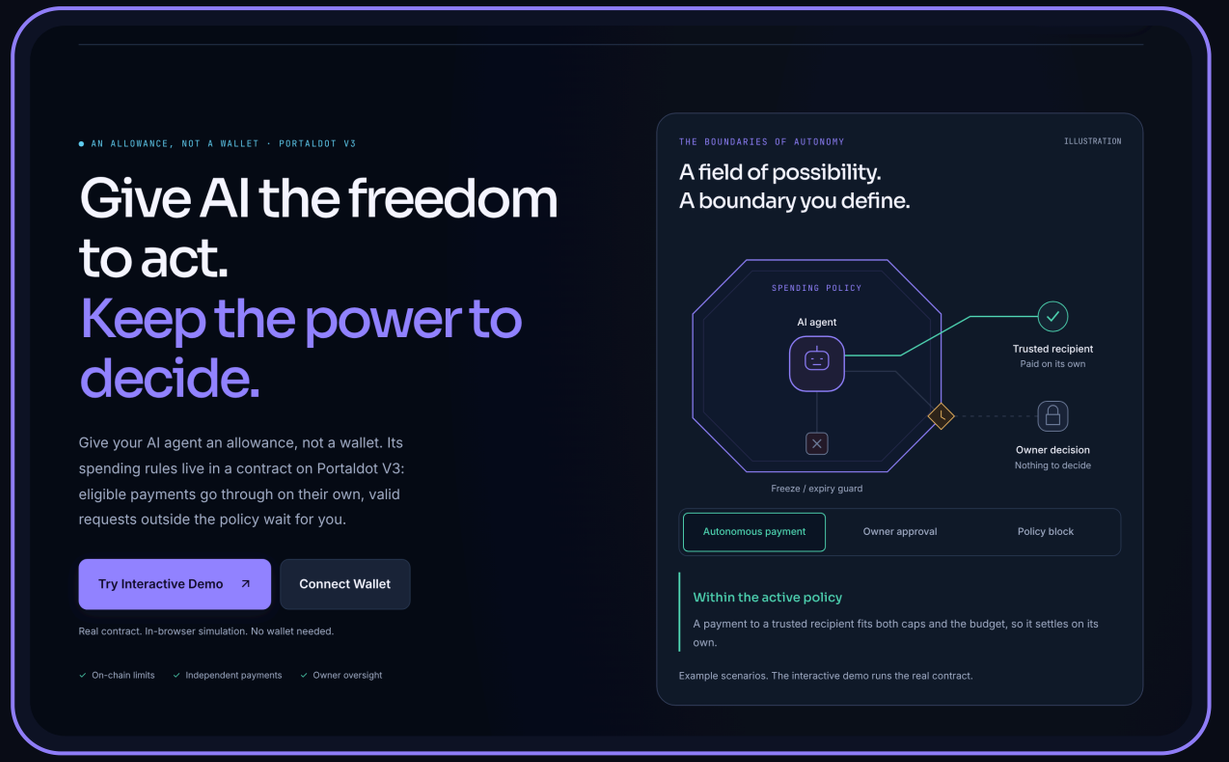
GIVE YOUR AI AGENT AN ALLOWANCE, NOT A WALLET.

On-chain spending control for autonomous AI agents on Portaldot V3.

Portaldot Hacker House 2026 · Track 4, Ecosystem DApps
Solo builder: Ozan Turan

RECORDED AI PAYMENTS | NATIVE + EVM OWNERS | TESTED ON-CHAIN



Working testnet prototype. Recorded testnet transactions. Open source (MIT).

58 Foundry | **118 MCP** | **24 native**

WHY THIS MATTERS

Autonomy without boundaries is a liability.

When an AI agent needs to pay, teams face an uncomfortable trade-off.

01

Unrestricted wallet key

The agent may spend funds held in its wallet beyond the intended task.

TOO MUCH TRUST

02

Approve every payment

Human review for each charge can erase the operational benefit of autonomy.

TOO MUCH FRICTION

03

App-level confirmation only

A UI confirmation is not the same as an enforceable on-chain spending budget.

WRONG ENFORCEMENT LAYER

Agents need programmable budgets enforced where the money actually moves.

THE CONTRACT

Rules on chain. Decisions only when needed.

A single allowance turns a broad agent key into bounded contract spending authority.

OWNER SETS POLICY — **Budget • payment cap • fixed window • trusted recipients • expiry**

● IN POLICY

PAID

An eligible payment settles autonomously from the allowance.

● VALID, OUTSIDE POLICY

HELD

The request waits for an owner decision; no funds are reserved.

● INVALID / FROZEN / EXPIRED

BLOCKED

The call fails. Freeze stops new agent operations at the contract.

The owner can approve or reject a held request, update rules, rotate the agent key, or freeze new agent spending.

BUILT AND WORKING

The owner stays in control.

Owner dashboard, shown here in its in-browser simulation: the real contract, demo data.



POLICY FIELD

A visual picture of what an agent may do.

SPENDING CAPACITY

Budget, per-payment cap and window headroom.

OWNER ACTIONS

Approval inbox, rule changes and freeze.

TRY WITHOUT A WALLET

A browser-local EVM runs the actual contract bytecode. Nothing is on a public chain.

ON-CHAIN PROOF / OCT 23

A real AI assistant. Real recorded transactions.

Claude used the Allowance MCP tools while the contract rules controlled each payment.

BLOCK 159822

0.3 POT PAID

Agent payment, within the caps

BLOCK 160197

2 POT HELD

Over the 1 POT cap

BLOCK 160231

OWNER FREEZE

New agent operations blocked

EXCERPT FROM THE RECORDED ASSISTANT SESSION · BLOCK 160197

“This time it went through to the approval queue. The 2 POT payment for one GPU hour is now waiting for your approval (request #2), because it’s over the 1 POT per-payment cap. Nothing moves until you approve it.”

```
recipient: 0x1807…8547 (the owner address)
memo: “Purchase of 1 GPU hour (2 POT), requested by user”
```

The record also shows the owner approving the held 2 POT request (block 160213).

A refusal after the freeze is recorded session behavior and produced no new transaction. The recipient was the owner address, so no external service delivery happened or was verified.

VIEW OCTOBER 23 RECORD ↗

PORTALDOT V3 / NOVEMBER 2, 2026

Native owner. EVM-key agent. One paid transaction.

01 NATIVE OWNER
5... account
Manual node-console signing

02 ALLOWANCE #1
0.1 POT / payment
0.5 POT/day · native-owner funded

03 CLAUDE / MCP
Agent EVM key
status → preview → pay

04 PAID EVENT
0.05 POT
Block 292133

8 native writes, signed by hand in the node console (blocks)

255537 mapAccount 282485 openAccount 291101 fund 291173 withdraw

291215 freeze 291242 unfreeze 291982 fund 292174 withdraw

Agent payment · block 292133

● **Paid 0.05 POT** · memo “native owner test” · confirmed in 3144 ms

`receipt: success · event: Paid · tx 0xe257c49b633924be013d5f54c48b3d39e97fbad4d73a5503b5c8e4981284d191`

RECORDED TESTNET RECEIPT

```
status: 1
event: Paid
```

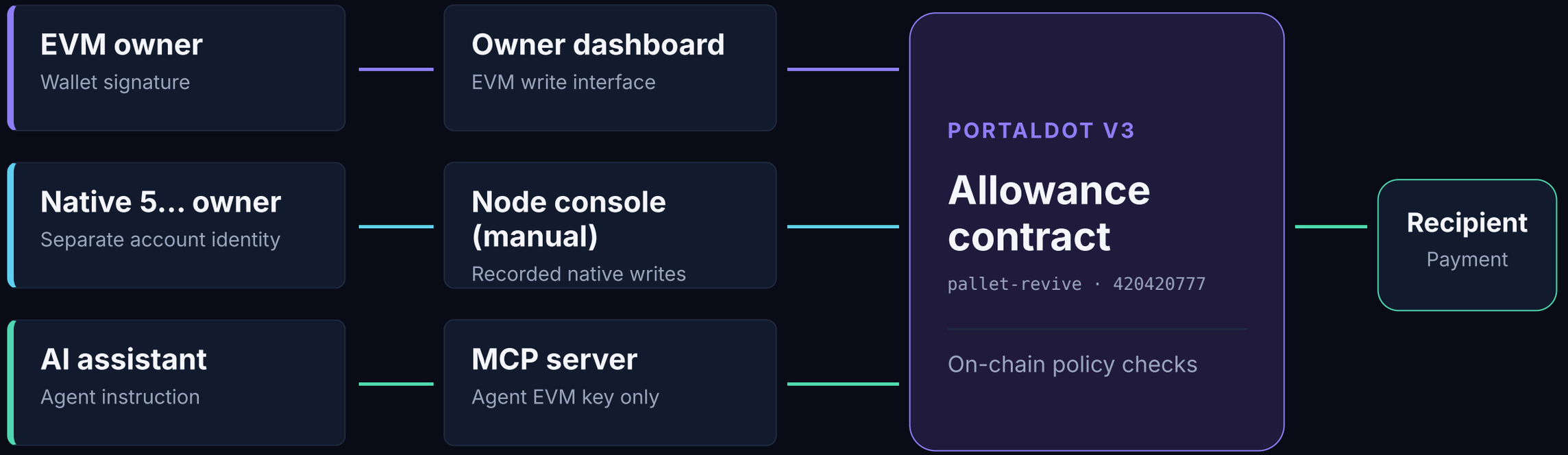
The recipient was the owner address, not a verified third-party purchase.

VIEW TX & FULL HASH ↗

HOW IT IS BUILT

A shared spending layer for owners and agents.

One contract, separate owner authority and agent execution paths.



The browser demo runs separately: same contract bytecode, no public-chain transaction.

ENGINEERING EVIDENCE

Tested beyond the happy path.

Implementation quality, adversarial cases and truthful confirmation semantics.

58

FOUNDRY TESTS

Fuzz (1,000 runs), invariants, freeze, expiry, approvals and payment edge cases.

118

MCP TESTS

Local-chain and protocol checks, receipt/event matching and uncertain outcomes.

24

NATIVE UNIT TESTS

Read-only identity inspection and failure handling; run locally.

176 contract + MCP tests run in CI • 24 native tests run locally

Internal analysis only. No independent audit. Testnet proof, not a production certification.

LEARNINGS FROM THE CHAIN

Interoperability tested. V3 storage deposits measured.

Evidence for Portaldot builders, including what still needs redesign.

NATIVE ↔ EVM UNITS

14 vs 18

decimals: the native console uses 14, EVM contracts use 18. Amounts must be scaled by hand.

Recorded fund / pay

STORAGE DEPOSITS

34.5735 POT

held when opening a native-owned allowance

3.8415 POT

per newly occupied storage word (observed)

Refunded only when the entire word returns to zero.

After the agent's first payment, the balance-word deposit stayed held (a v2 redesign item). Historical testnet measurements, not fee guarantees.

WHY PORTALDOT, AND WHY KEEP SUPPORTING IT

Built for V3. Useful to the ecosystem.

What Allowance needs from Portaldot, and what the ecosystem gets back.

WHY PORTALDOT IS ESSENTIAL HERE

1 Solidity runs on V3 unchanged: the same contract bytecode runs in the browser demo and on the testnet.

2 Native and EVM identities meet in one flow: a native 5… account owned an allowance and an EVM-key agent paid from it (block 292133).

3 POT is both the budget and the gas asset, with ERC-20 budgets supported by the same contract.

WHY IT DESERVES CONTINUED SUPPORT

1 Measured V3 findings are published for other builders: 14 vs 18 decimals, 3.8415 POT per storage word, 34.5735 POT at opening.

2 The next steps are concrete: v2 storage layout, crash-recovery wiring, an independent audit, pilot integrations.

3 Agents make many small payments, a usage pattern we expect to matter for the ecosystem (a hypothesis, not measured).

Support we would apply for: architecture review, audit guidance and introductions to AI-agent teams.

FROM PRODUCT TO USE

Open infrastructure. A commercial hypothesis to test.

The core is open source; customer demand and willingness to pay remain unvalidated.

WHO NEEDS THIS

Agent builders

Teams running bots and autonomous agents that need bounded purchasing authority.

WHAT SHIPS NOW

Open core

MIT-licensed smart contract, owner dashboard and MCP server.

POSSIBLE PAID LAYER

Teams

Hosted runners, reporting, alerts and approval workflows. Not built yet.

THE NEXT VALIDATION LOOP

- 01 Interview AI developers
- 02 Pilot with integrations
- 03 Measure repeated usage
- 04 Test willingness to pay

FROM TESTNET PROOF TO INFRASTRUCTURE

Autonomous payments.
Bound by your rules.

The technology works on the testnet. Production use is a no-go for now.

1	**OPERATIONAL SAFETY**	Journal integration, idempotency, nonce safety, independent finality checks.
2	**UNIT ECONOMICS**	Frequent-use cost model and v2 storage/refund redesign.
3	**REAL-WORLD VALIDATION**	Pilot integrations, demand testing and independent security review.

TRY ALLOWANCE

Scan for the demo

OPEN SOURCE / MIT

Real tx proof

VIEW EVIDENCE ↗

VIDEO (2:56) ↗ DEMO ↗ GITHUB ↗ PROOF ↗